

Module IV
CLOUD COMPUTING

NEUTRON

What is Neutron?

What can Neutron do?

Basic Neutron Architecture

- Neutron API Server
- Core Plugin :Network, Port, Subnet
- Service Plugins
- Agents :L2 Agent, DHCP Agent, L3 Agent, VPN Agent
- API Extensions

IMPLEMENTING VIRTUAL NETWORKS

- VLAN-based networks
- Tunnel-based networks
- Virtual switches
- The ML2 plugin
- Network types
- Neutron subnets

CONNECTING VIRTUAL NETWORKS WITH ROUTERS

- Configuring the routing service
- Connecting networks using a virtual router
- Connecting to the external world
- Providing connectivity from the external world
- Associating a floating IP to a virtual machine

IMPLEMENTING NETWORK SECURITY IN OPENSTACK

Security groups

- Creating security group policies

Firewall as a service

- Configuring the firewall service
- Creating firewall policies and rules

Neutron

What is Neutron?

Neutron is the networking service of OpenStack.

Neutron connects virtual machines (VMs) to networks and allows them to communicate with each other and with the outside world.

[**OpenStack** is an open-source cloud computing platform used to create and manage a private or public cloud.

OpenStack helps an organization create its own cloud environment where users can create virtual machines, storage, networks, and other cloud resources.

It is similar to the cloud infrastructure used by services such as AWS, Azure, or Google Cloud, but OpenStack is open-source and can be deployed by organizations on their own hardware.

Eg: Imagine a college has 20 physical servers in its data center.

Instead of giving one complete physical server to each department, the college installs OpenStack.]

A virtual machine is a computer created using software inside another physical computer.

It behaves like a real computer. It can have its own:

- Operating system
- CPU resources
- RAM
- Storage
- Network connection
- Applications

Example

Suppose a college has one powerful physical server:

- The physical server can create **multiple virtual machines**.
- For example:
- **VM1** → Ubuntu for MCA students
- **VM2** → Windows for programming
- **VM3** → Linux for research

Although they are separate virtual computers, they are actually using resources from the **same physical server**.

- **Physical machine = Real computer**
- **Virtual machine = Computer created using software**
- **OpenStack = Platform that manages these virtual machines and other cloud resources.**

Real-life example

- Suppose a college has 100 computers.

They need:

- A network to communicate with each other
- IP addresses
- Routers to access the Internet
- Firewall for security

Similarly, in OpenStack, many **virtual machines** need:

- Virtual networks
- IP addresses
- Routers
- Security
- Internet connectivity

Neutron provides these networking facilities.

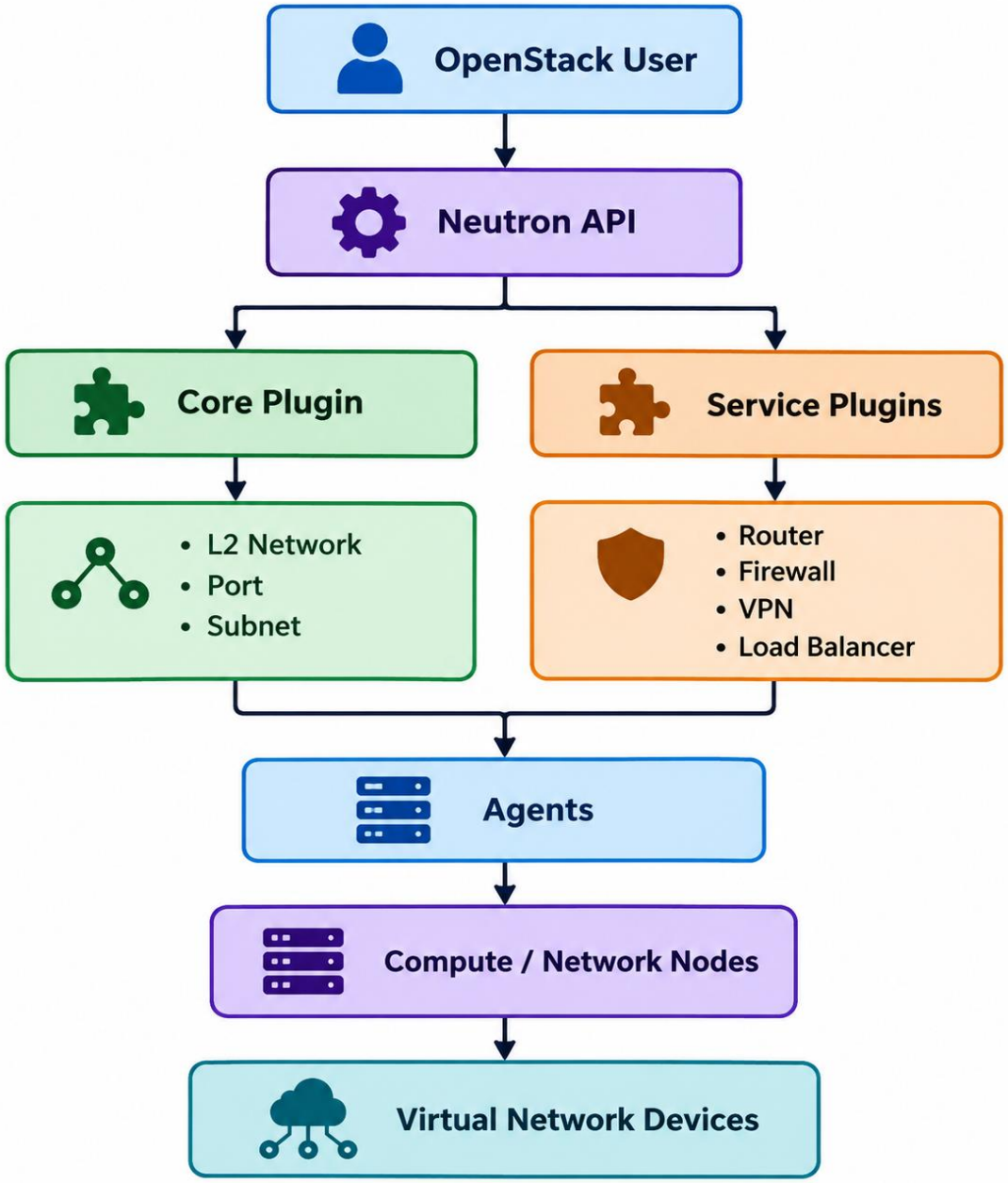
What can Neutron do?

Neutron can provide:

Function	Meaning	Example
Switching	Connect VMs in the same network	VM1 ↔ VM2
Routing	Connect different networks	Network A ↔ Network B
DHCP (Dynamic Host Configuration Protocol)	Automatically assign IP addresses	VM gets 192.168.1.10
Firewall	Protect the network	Block unwanted traffic
VPN	Secure connection between networks	Office ↔ Cloud
Load Balancer	Distribute traffic	Web server 1, 2, 3
Floating IP	Give external access to a VM	Internet → VM

So, Neutron helps create a **complete virtual network topology**.

- Basic Neutron Architecture



Main components

Neutron mainly consists of:

- **Neutron API Server**
- **Core Plugin**
- **Service Plugins**
- **Agents**
- **API Extensions**

Neutron API Server

- The **Neutron API Server** is the **entry point** for networking requests.
- Users or OpenStack services send networking requests to the API.

Example

- Suppose a user wants to create a network called :College - Network
- The request goes to:

User

↓

Neutron API

↓

Plugin

↓

Network created

- The API receives the request and sends it to the appropriate plugin.
- The API server is generally deployed on the **OpenStack controller node**. Multiple API server instances can be deployed for scalability and availability.

Plugin

A plugin is a software component that tells Neutron how to implement a networking function.

Think of a plugin as a translator or manager.

Neutron API

↓

Plugin

↓

Networking resources

Neutron uses a plugin-based architecture, which gives flexibility.

There are two major types:

- Core Plugin
- Service Plugin

- **Core Plugin**
- The Core Plugin mainly provides Layer-2 (L2) networking.
- It connects virtual machines and network devices to a virtual network.
- The core plugin manages important resources such as:

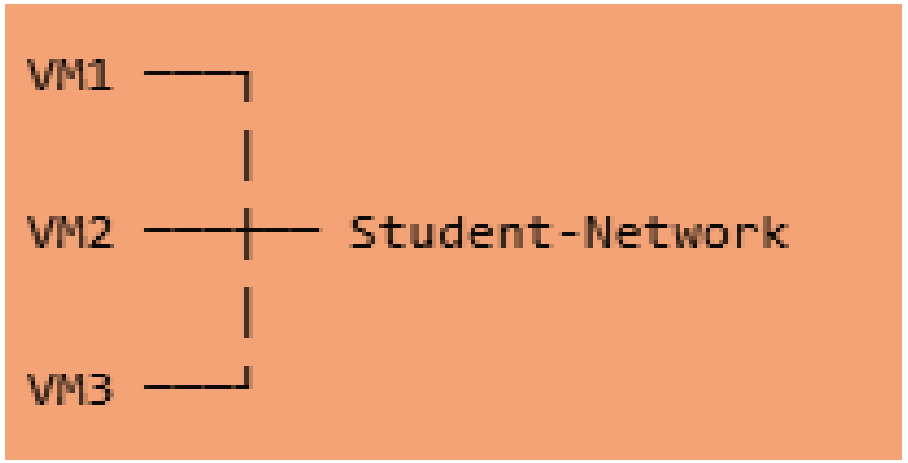
a) Network

A network represents a virtual Layer-2 network.

Example:

Student-Network

Several VMs can connect to this network.

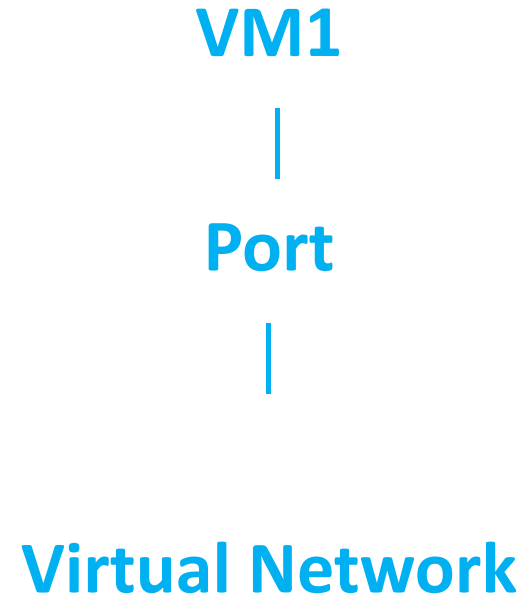


b) Port

A port is a connection point on a network.

Think of a physical computer's LAN port.

For example:



A VM connects to a Neutron network through a port.

c) Subnet

A subnet provides the IP address range for a network.

Example:

Network: Student-Network

Subnet:192.168.1.0/24

VMs may receive:

VM1 → 192.168.1.10

VM2 → 192.168.1.11

VM3 → 192.168.1.12

- **Service Plugin**

A Service Plugin provides higher-level networking services.

Examples: Routing, Firewall, VPN, Load Balancing

Core Plugin

↓

Basic network connectivity

↓

Service Plugin

↓

Advanced network services

- **Agents**

Agents are software processes that actually perform networking tasks on compute and network nodes.

Think of:

  - Plugin = Manager
  - Agent = Worker

- The plugin decides what should be done, and the agent helps implement it on the actual node.
- Agents communicate with the Neutron server using RPC (Remote Procedure Call) through the message bus.

Types of Neutron Agents

Important agents include:

L2 Agent

The L2 agent provides Layer-2 connectivity.

It connects:

VM

↓

Virtual Switch

↓

Network

It is generally present on compute/network nodes as required by the deployment.

Example:

A VM is running on Compute Node 1.

The L2 agent helps connect that VM to the correct virtual network.

DHCP Agent

- The DHCP agent provides DHCP service.



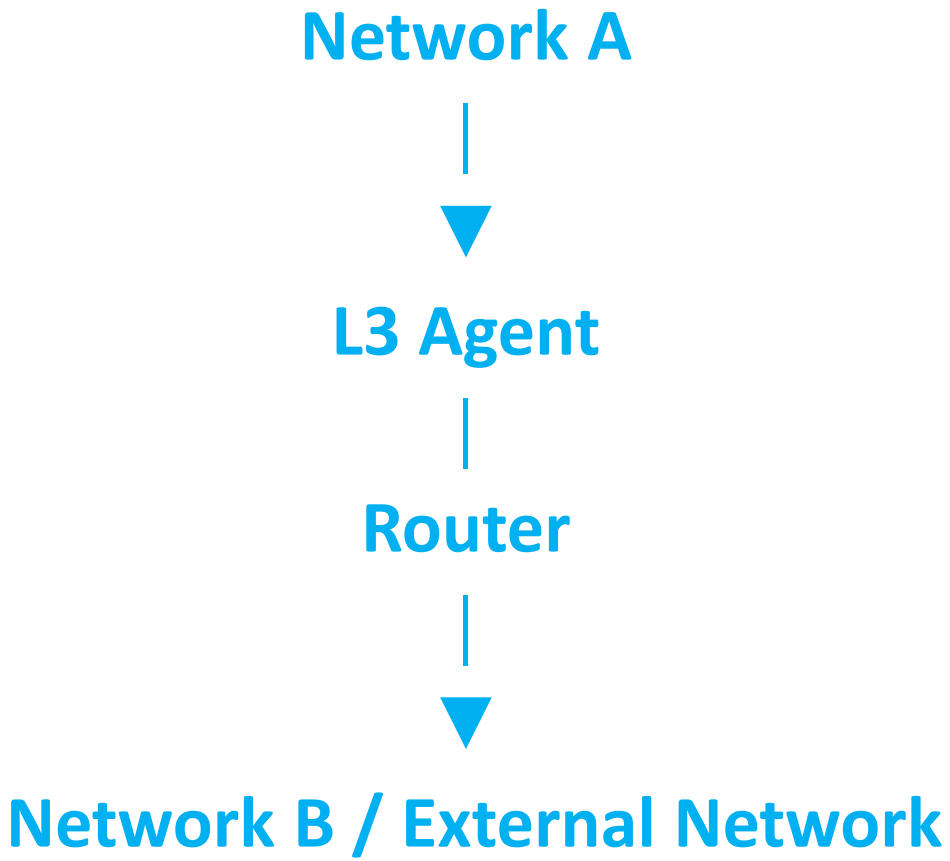
Example:

VM requests an IP address.

DHCP agent provides:192.168.1.20

L3 Agent

- The L3 agent provides routing and NAT services.

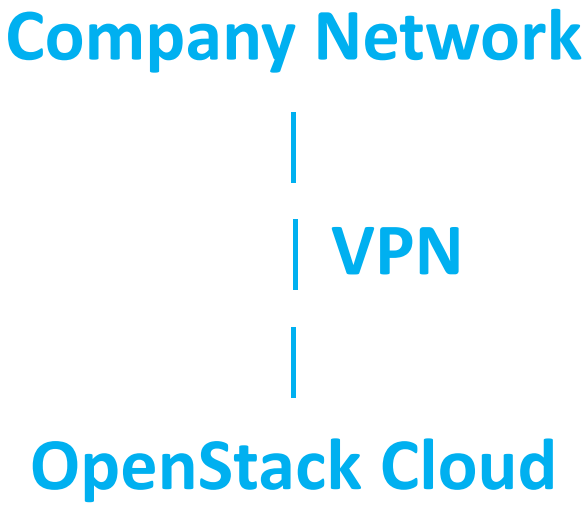


- It helps provide connectivity between networks and external access.

VPN Agent

- The VPN agent provides VPN functionality.

Example:



It can create a secure connection between networks.

Neutron API Extensions

- API extensions allow Neutron to provide additional or advanced networking features through the API.
- API extensions add extra capabilities to the basic Neutron API.
- For example, basic Neutron provides network, subnet and port operations.
- Additional functionality can expose features such as:
  - Security groups
  - Floating IPs
  - Advanced networking services
  - Other extended networking resources

Example

Basic API:

Create Network

Create Subnet

Create Port

Extended API:

Create Floating IP

Configure Security Group

Configure Advanced Service

Thus, extensions make Neutron more flexible and extensible.

IMPLEMENTING VIRTUAL NETWORKS

- VLAN-based networks
- Tunnel-based networks
- Virtual switches
- The ML2 plugin
- Network types
- Neutron subnets

Implementing Virtual Networks in OpenStack Neutron

In OpenStack, **Neutron** is responsible for creating and managing virtual networks for virtual machines (VMs). A virtual network allows VMs to communicate with each other, with other networks, and with the external/Internet network.

A simple real-life example is a **college campus network**:

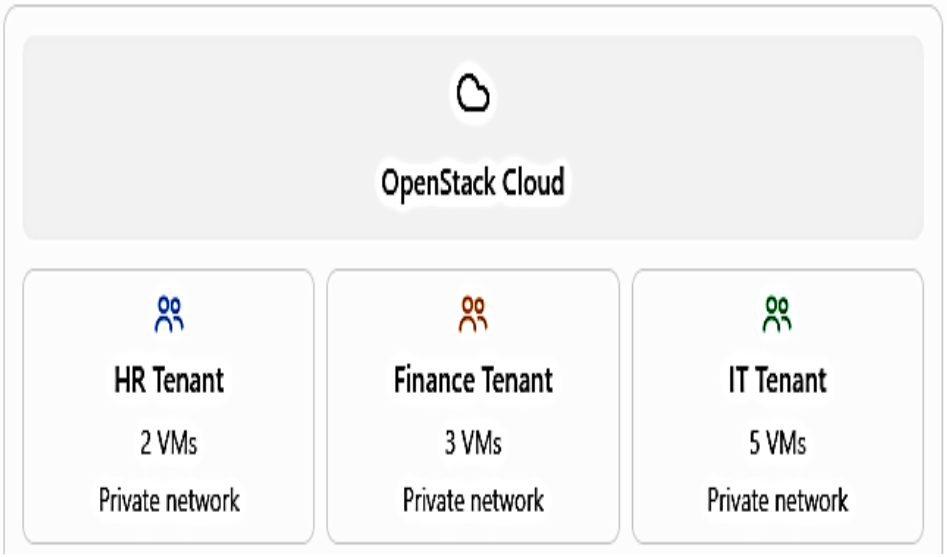
- Different departments can have separate networks.
- The Computer Science department may have one network.
- The MCA department may have another network.
- A router can connect these networks.
- A firewall can control communication.
- VLANs or tunnels can keep network traffic separated.

VLAN-based and tunnel-based networking are two methods used to isolate tenant networks and enable communication between VMs.

- In OpenStack, a **tenant is a group or an isolated environment that owns and uses cloud resources**. In modern OpenStack terminology, it is generally called a **project**.
- Think of a tenant as a customer or department that has its own resources inside a shared cloud.

Real-life example

- Imagine a company has an OpenStack cloud shared by three departments.



Each tenant can have its own:

- Virtual machines (VMs)
- Networks and subnets
- Routers and security groups
- Storage volumes
- Users and access permissions

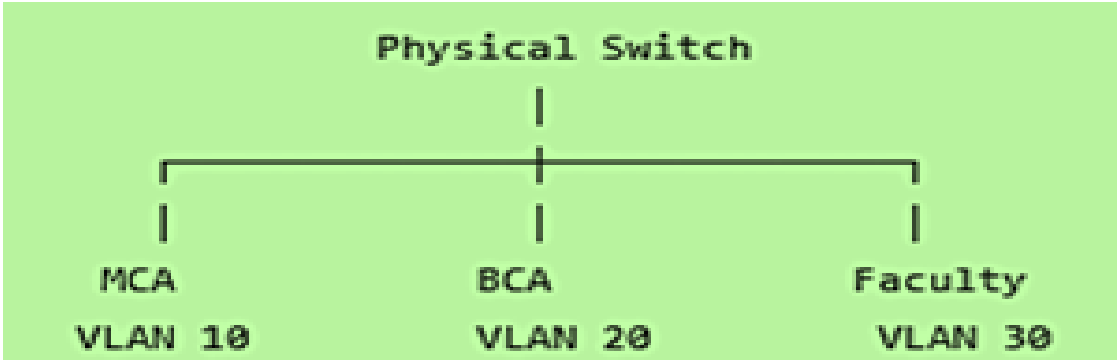
VLAN-Based Networks

What is VLAN?

- **VLAN = Virtual Local Area Network**
- A VLAN divides one physical network into multiple **logical networks**.
- Even though computers may use the same physical switches and cables, VLANs allow them to behave as if they are on separate networks.

Example

- Suppose a college has one physical switch:



- MCA computers belong to VLAN 10, BCA computers to VLAN 20, and faculty computers to VLAN 30.
- The physical infrastructure is shared, but the networks are logically separated.

- **VLAN ID**

Each VLAN is identified by a **VLAN ID**, normally from **1 to 4094** for IEEE 802.1Q VLANs.

For example:

Department	VLAN ID
MCA	10
BCA	20
Faculty	30

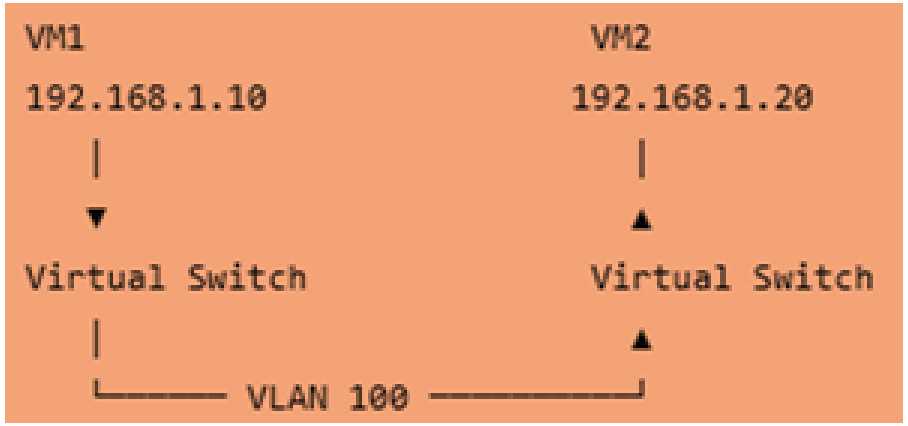
Traffic belonging to VLAN 10 is logically separated from traffic belonging to VLAN 20.

How VLAN Networking Works

- **VLAN (Virtual LAN)** is used to divide one physical network into separate **logical networks**.

Example

- Suppose we have two VMs:
- Both VM1 and VM2 belong to **VLAN 100**.



When VM1 sends data to VM2:

- **Step 1:** VM1 creates the data packet.
- **Step 2:** The packet goes to the **virtual switch**.
- **Step 3:** The network identifies the traffic as belonging to **VLAN 100**.
- **Step 4:** The physical network carries the VLAN 100 traffic.
- **Step 5:** The destination virtual switch receives it.
- **Step 6:** The virtual switch sends the data to VM2.

VLAN (Virtual Local Area Network) networking uses VLAN IDs to separate network traffic over shared physical network infrastructure.

How it works

- Each tenant network is assigned a VLAN ID, typically between 1 and 4094.
- Network traffic is tagged with the appropriate VLAN ID using IEEE 802.1Q.
- Physical switches and network interfaces forward traffic based on VLAN tags.
- VMs belonging to the same VLAN can communicate with each other, subject to network configuration and security rules.

Example: Tenant A uses VLAN 100, while Tenant B uses VLAN 200. Their traffic remains isolated even if it travels over the same physical switch.

Advantages of VLAN

- Network isolation
- Different tenants or departments can be separated.
- Efficient use of physical network
- Multiple logical networks can share the same physical infrastructure.
- Good performance
- VLAN networking generally uses the physical network directly.
- Simple for data centers that already use VLANs

Disadvantages of VLAN

- VLAN configuration is required on physical switches.
- VLAN IDs are limited.
- Large-scale multi-tenant environments can become difficult to manage.
- Physical network configuration must match the OpenStack configuration.

- **Imagine a company with 3 departments**
- A company has an OpenStack cloud running virtual machines for three departments:
- HR department
- Finance department
- IT department
- Each department has its own VMs, and the company wants to keep their network traffic separate.
- Imagine the company uses physical network switches that support VLANs.

Department	VM	VLAN ID
HR	HR-VM1	10
Finance	Finance-VM1	20
IT	IT-VM1	30

How it works:

- HR-VM1 sends a network packet.
- The physical or virtual switch associates the traffic with VLAN 10.
- The network forwards the traffic within VLAN 10.
- Finance traffic uses VLAN 20, so it remains separated from HR traffic.
- In OpenStack, the network administrator configures the Neutron VLAN network and maps it to a physical network that supports the corresponding VLAN.

Real-life analogy: Think of a building with three departments using separate lanes in the same corridor. Each lane is identified by a number, and people are directed to the correct lane.

- **Tunnel-Based Networks**
- A **tunnel-based network** creates a logical connection between network nodes by encapsulating one network packet inside another packet.
- Think of it like a **private tunnel through a public road**.
- **Real-life example**
- Suppose two buildings are far apart:

```
Building A                          Building B
Computer ——— Private Tunnel ——— Computer
```

The traffic travels through an existing physical network, but the tunnel keeps the logical network separated.

Why does OpenStack use tunnels?

OpenStack clouds may have many tenants.

For example:

- Tenant A → Network A
- Tenant B → Network B
- Tenant C → Network C

All of them may use the same physical network

Tunnelling provides logical isolation between these networks.

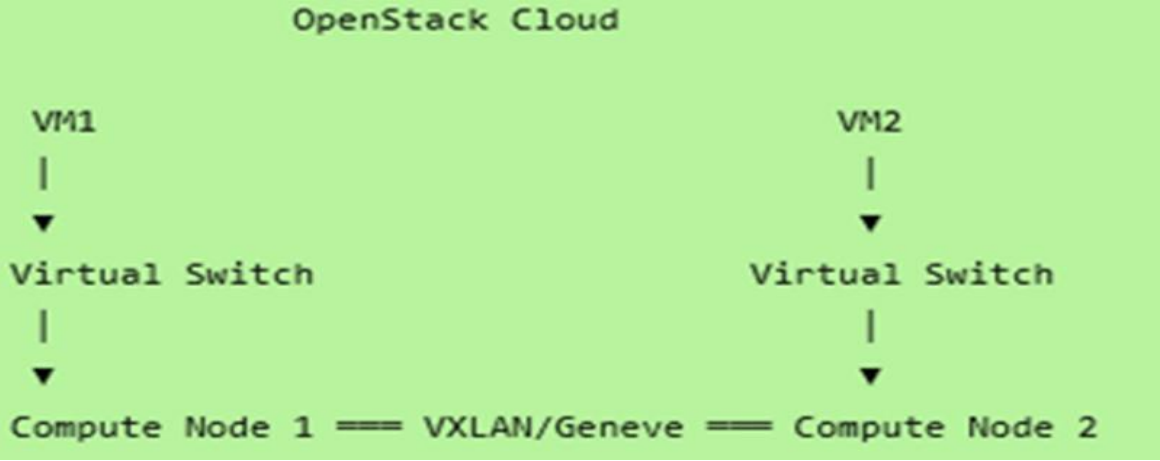
Common tunnel technologies

OpenStack Neutron commonly supports technologies such as:

- **VXLAN** : It is widely used in cloud environments because it supports a very large number of logical network segments.
- **GRE** : GRE creates tunnels between network endpoints and can carry different types of network traffic.
- **Geneve :** Geneve is a flexible network virtualization encapsulation protocol designed for modern cloud environments.

Tunnel example

Suppose VM1 is on Compute Node 1 and VM2 is on Compute Node 2.



The VMs can communicate even though they are running on different physical servers.

Advantages of tunnel-based networks

- Does not require one physical VLAN for every virtual network
- Suitable for large cloud environments
- Virtual networks can span multiple physical hosts
- Provides network virtualization over an existing IP network

Disadvantages

- Encapsulation adds some overhead.
- Requires additional configuration.
- Troubleshooting can be more complex.
- Network performance may depend on the underlying physical network.

- Now imagine the company has 10 compute servers spread across different racks. They are connected by an IP network, and the company wants to create hundreds of isolated tenant networks without configuring a separate physical VLAN for each one.
- OpenStack can use VXLAN tunnels.

Department	VM	VXLAN Network ID (VNI)
HR	HR-VM1	5001
Finance	Finance-VM1	5002
IT	IT-VM1	5003

How it works:

- Suppose HR-VM1 is running on Compute Node 1 and HR-VM2 is running on Compute Node 8.
- HR-VM1 sends a packet to HR-VM2.
- OpenStack's virtual networking layer encapsulates the packet in a VXLAN packet identified with VNI 5001.
- The encapsulated packet travels over the IP network between the compute nodes.
- Compute Node 8 decapsulates the packet and delivers it to HR-VM2.
- Finance traffic uses VNI 5002 and remains logically isolated from the HR network.

Real-life analogy: Imagine sending a letter inside a courier envelope. The outer envelope carries the address needed to deliver it across the city, while the letter inside contains the original message. VXLAN is similar: it wraps the original network packet in an outer packet for transport.

Virtual Switches

- A **virtual switch** is a software-based network switch.
- It connects **virtual machines (VMs)** to a virtual network.
- It performs a function similar to a **physical Ethernet switch**.
- In OpenStack, virtual switches are commonly used to provide network connectivity to VMs.
- Virtual switches connect virtual network ports to the network.
- They are installed on compute nodes and network nodes.
- VMs running on compute nodes connect to virtual switches.
- Network elements such as:
  - Virtual routers
  - DHCP services
  - Other virtual network devices can also connect to virtual switches.
- The virtual switch connects to the physical network through the server's NIC (Network Interface Card).
- A commonly used virtual switch technology is Open vSwitch (OVS).

The ML2 Plugin

- **ML2** stands for **Modular Layer 2**.
- It is a major plugin framework in OpenStack Neutron.
- It allows Neutron to support different types of Layer-2 networking technologies.
- It makes networking **flexible and modular**.

Why is ML2 needed?

- Different organizations may use different networking technologies such as:
  - VLAN
  - VXLAN
  - GRE
  - Geneve
  - Flat networks
- ML2 allows OpenStack to work with these different technologies through a common framework.

Main components of ML2

ML2 mainly uses two important types of drivers:

1. Type Drivers

They define **WHAT type of network** is being used.

Examples:

- Flat
- VLAN
- VXLAN
- GRE
- Geneve

2. Mechanism Drivers

They define **HOW the network is implemented**.

Examples:

- Open vSwitch
- Linux Bridge
- SR-IOV

Network Types

OpenStack Neutron supports different network types.

1. Flat Network

- A **Flat network** is a simple network without VLAN tagging.
- All VMs connected to the network share the same physical network segment.
- It is simple to configure.

2. VLAN Network

- VLAN stands for **Virtual Local Area Network**.
- It divides one physical network into multiple logical networks.
- Each VLAN has a **VLAN ID**.

3. VXLAN

- VXLAN stands for **Virtual Extensible LAN**.
- It is an **overlay network technology**.
- It allows virtual networks to communicate across different physical hosts.
- It is useful in large cloud environments.

4. GRE

- GRE stands for **Generic Routing Encapsulation**.
- It creates a tunnel between network endpoints.
- Network packets are encapsulated and transported through the physical network.

5. Geneve

- Geneve stands for **Generic Network Virtualization Encapsulation**.
- It is another overlay networking technology.
- It is designed to support flexible network virtualization.
- It can carry additional metadata with network traffic.

Neutron Subnets

- A subnet is a logical division of a network used for IP addressing.
- In OpenStack Neutron, a subnet is associated with a Neutron network.
- It provides the IP address range from which VMs can receive IP addresses.
- Network vs Subnet
- Network = Layer-2 connectivity
- Subnet = Layer-3 IP addressing

Components of a subnet

Network Address

- Defines the beginning of the IP network.
- Example:192.168.10.02.

CIDR (Classless Inter-Domain Routing)

- CIDR is a method used to specify an **IP network and its size**..
- Example:192.168.10.0/24
- A /24 IPv4 network contains 256 total addresses, with 254 normally usable host addresses.

Gateway

The gateway allows VMs to communicate with other networks.

Example:192.168.10.14.

DHCP

DHCP can automatically provide IP configuration to VMs.

Allocation Pool

It specifies the range of IP addresses that can be automatically assigned.

CONNECTING VIRTUAL NETWORKS WITH ROUTERS

- Configuring the routing service
- Connecting networks using a virtual router
- Connecting to the external world
- Providing connectivity from the external world
- Associating a floating IP to a virtual machine

Connecting Virtual Networks with Routers in OpenStack Neutron

In OpenStack, **Neutron** is responsible for networking. A virtual machine (VM) may need to communicate with:

- Another VM in the same network
- A VM in another network
- The Internet
- Users coming from the Internet

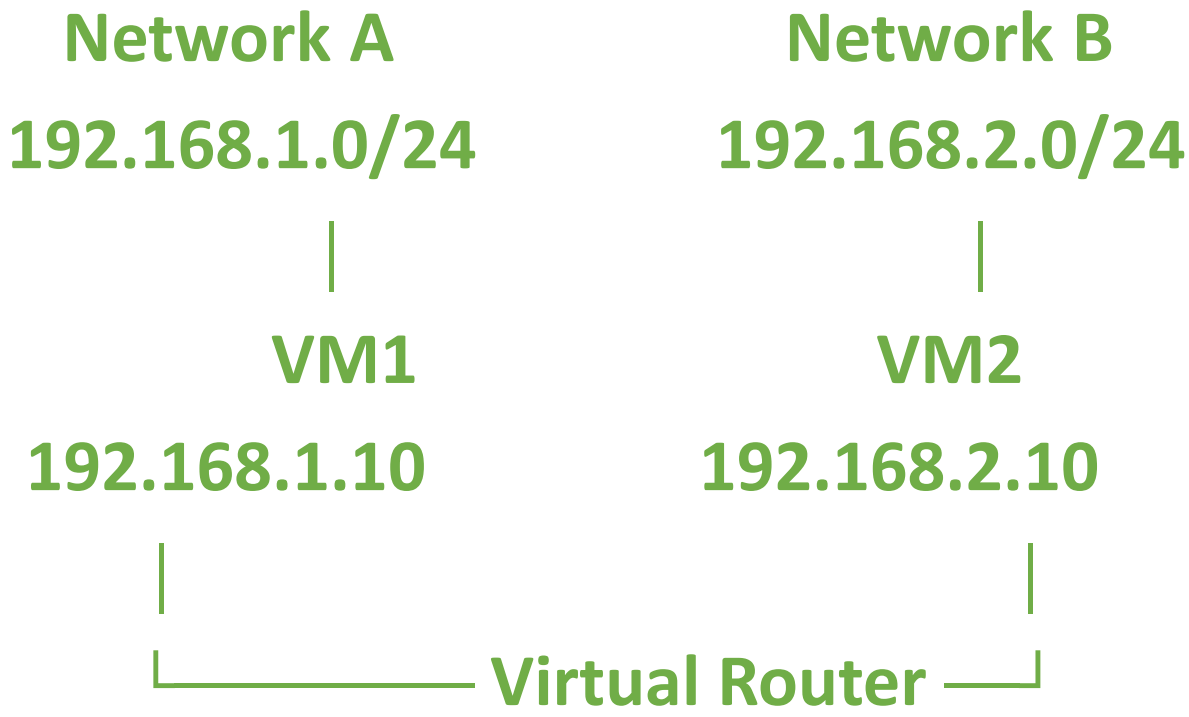
For these purposes, we use **virtual routers**.

Configuring the Routing Service

- **Routing**

Routing means transferring data from one network to another network.

For example:



VM1 and VM2 belong to different networks. The **virtual router** allows them to communicate.

In OpenStack

Neutron provides routing through its Layer 3 (L3) service.

Important components are:

- **L3 Service Plugin** – provides routing functionality.
- **Virtual Router** – logically connects different networks/subnets.
- **L3 Agent** – implements routing functions on the appropriate OpenStack node.
- **Router Interface** – connects a subnet to the router.
- **External Gateway** – connects the router to an external network.

Steps to configure routing

- Create the first virtual network.
- Create a subnet for the network.
- Create the second virtual network.
- Create a subnet for the second network.
- Create a virtual router.
- Add the first subnet to the router.
- Add the second subnet to the router.
- Now the router can forward traffic between the two networks.

Example

Consider a college:

MCA Network → 192.168.1.0/24

MBA Network → 192.168.2.0/24

The college router connects these two networks.

Similarly:

OpenStack Virtual Router = College Network Router

Connecting Networks Using a Virtual Router

- A **virtual router** is a software-based router created and managed by OpenStack Neutron.
- It can connect two or more virtual networks.

Example

- Suppose we have:

MCA Network

192.168.1.0/24

|

Virtual Router

|

MBA Network

192.168.2.0/24

VM1:IP = 192.168.1.10

VM2:IP = 192.168.2.10

- Because they are in different networks, they need a router to communicate.



Steps

- Create Network A.
- Create Subnet A.
- Create Network B.
- Create Subnet B.
- Create a virtual router.
- Add Subnet A as a router interface.
- Add Subnet B as another router interface.
- The router now connects both networks.

EXAMPLE:

Imagine a college with:

- Computer Lab network
- Faculty network
- Students and faculty are on separate networks for security.
- The college router allows selected communication between them.

Similarly:

Virtual Router connects separate OpenStack virtual networks.

Connecting to the External World

OpenStack VMs are normally placed in **private networks**.

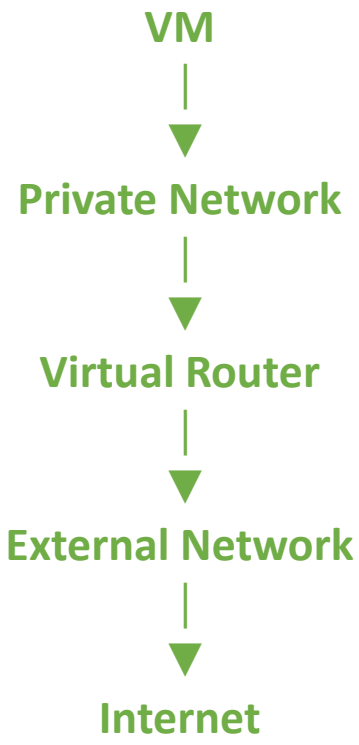
For example:

Private Network 192.168.10.0/24

A VM may have: VM1 = 192.168.10.10

- This is a **private IP address**.
- A private IP normally cannot be directly reached from the public Internet.
- Therefore, we connect the private network to an **external network** using a virtual router.

Basic structure



Example

Think about your college computer.

College Computer
↓
College LAN
↓
College Router
↓
ISP
↓
Internet

OpenStack works similarly:

VM
↓
Private Network
↓
Virtual Router
↓
External Network
↓
Internet

STEPS

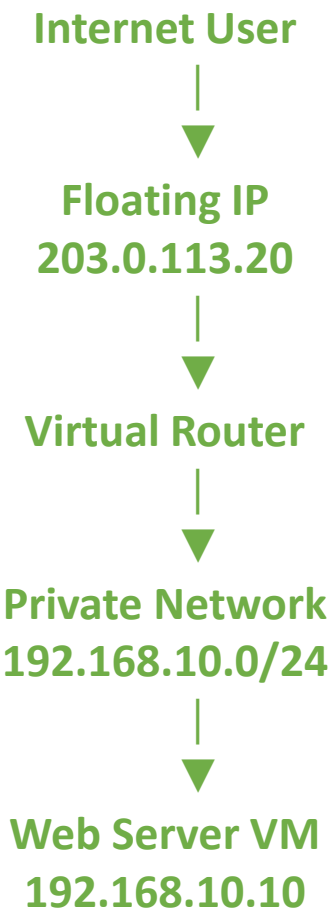
- Create a private network.
- Create a subnet.
- Create a virtual router.
- Connect the private subnet to the router.
- Configure the router's external gateway.
- The router becomes connected to the external network.
- VMs can now communicate with external networks, depending on routing and security configuration.

The **external gateway** provides the connection between the virtual router and the external network.

Providing Connectivity from the External World

- This is the opposite direction.
- Here, an external user wants to access a VM inside OpenStack.
- For example, suppose a college hosts a web server in OpenStack.
- The web server has a private IP: 192.168.10.10
- An Internet user cannot normally directly access this private address.
- Therefore, OpenStack can provide a Floating IP.

Communication



Floating IP acts as an externally reachable address associated with the VM.

Example

Suppose your college has a web server.

Inside the college:

Web Server = 192.168.10.10

The college provides a public address:

Public Address = 203.0.113.20

People outside the college use the public address to reach the web server.

Similarly, OpenStack uses a **Floating IP** to provide external access to a VM.

Security

External connectivity should also be controlled using **security groups/firewall rules**.

For example:

Allow HTTP → Port 80

Allow HTTPS → Port 443

Allow SSH → Port 22 only when required

Associating a Floating IP to a Virtual Machine

- **Floating IP**
- A **Floating IP** is an IP address from an external network that can be associated with a virtual machine.
- It allows the VM to be reachable from outside the private network, subject to routing and security rules.
- **Example**
- VM has: Private IP: 192.168.10.10
- Floating IP:203.0.113.20
- OpenStack creates a mapping conceptually like:

203.0.113.20 → 192.168.10.10

Floating IP VM private IP

The virtual router performs the required address translation/routing.

Steps to associate a Floating IP

Step 1: Create an external network

Step 2: Create a private network

Step 3: Create a private subnet

Step 4: Create a virtual router

Step 5: Connect the private subnet to the router

Step 6: Set the external gateway

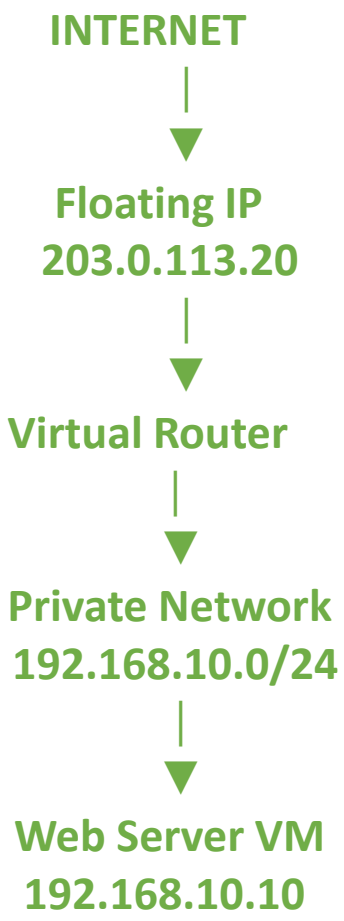
Step 7: Create/allocate a Floating IP

Step 8: Associate the Floating IP with the VM

Now external users can use the floating address to reach the VM, if security rules permit it.

Example: College Web Server

Suppose a college wants to host its website on an OpenStack VM.



What happens?

- College creates a VM.
- VM receives private IP 192.168.10.10.
- VM is connected to a private network.
- A virtual router is created.
- The private network is connected to the router.
- Router is connected to an external network.
- Floating IP 203.0.113.20 is allocated.
- Floating IP is associated with the VM.
- An external user sends a request to 203.0.113.20.
- Neutron's networking components route the traffic toward the VM.
- Security rules determine whether the traffic is allowed.
- The web server responds to the user.

IMPLEMENTING NETWORK SECURITY IN OPENSTACK

Security groups

- Creating security group policies

Firewall as a service

- Configuring the firewall service
- Creating firewall policies and rules

Implementing Network Security in OpenStack

A. Security Groups

1. Meaning of Security Group

- A **Security Group** is a collection of network security rules.
- It controls the traffic going **into and out of a Virtual Machine (VM)**.
- It works like a **security guard for a VM**.
- It can allow or restrict traffic based on:
  - IP address
  - Protocol
  - Port number
  - Traffic direction

2. Types of Traffic

- **Inbound traffic**
- Traffic coming **into** the VM.
- Example: User accessing a web server.

  Internet → VM
- **Outbound traffic**
- Traffic going **out of** the VM.
- Example: VM accessing a website.

  VM → Internet

Creating Security Group Policies

Steps

- Create a security group.
- Give the security group a name.
- Add security rules.
- Specify the protocol.
- Specify the port.
- Specify the source or destination.
- Apply the security group to the VM/port.
- Test the connection.

Example

- A college web server may allow:
- Students → Website → Port 80/443 ✓
- Administrator → Server → SSH Port 22 ✓
- Unauthorized traffic → Server ✗

Firewall as a Service (FWaaS)

- FWaaS provides **firewall functionality through OpenStack networking**.
- A **firewall** is a **network security system that monitors and controls incoming and outgoing network traffic** based on predefined security rules
- It controls network traffic using firewall rules.
- It can allow or deny traffic.
- It provides an additional layer of network security.

Example : A **firewall** is like a **security guard at the entrance of a college**. It checks network traffic and decides whether to **ALLOW or BLOCK** it.

Configuring the Firewall Service

Steps

- Enable/configure the firewall service in the OpenStack environment.
- Create firewall rules.
- Define what traffic should be allowed.
- Define what traffic should be blocked.
- Create a firewall policy.
- Add firewall rules to the policy.
- Apply the policy to the appropriate supported network/firewall resource.
- Test the firewall configuration.
- **Example**
- For a web server:
- HTTP → Port 80 → ALLOW

  HTTPS → Port 443 → ALLOW

  SSH → Port 22 → ALLOW from admin network

  Other unwanted traffic → BLOCK

Creating Firewall Policies and Rules

1. Firewall Rule

- A **firewall rule** is one individual instruction.
- It specifies how a particular type of traffic should be handled.
- It can specify:
  - Protocol
  - Source
  - Destination
  - Port
  - Action

2. Firewall Policy

- A **firewall policy** is a collection of firewall rules.
- Rules are arranged in a defined order.
- The policy is applied to the appropriate firewall resource.

Security Group and Firewall Together

- Both can be used as different layers of protection.



Security Group → Controls traffic associated with VM interfaces/ports.

FWaaS → Controls network-level traffic using firewall policies/rules.